Microservices Architecture Design

EasyParkPlus Parking Management System

1. ARCHITECTURE OVERVIEW

1.1 Design Principles

Based on Michael's requirements and the DDD analysis, the architecture follows:

- **Bounded Context per Service:** Each bounded context becomes a microservice
- **Database per Service:** Each service owns its data
- **Event-Driven Communication:** Services communicate via events
- **API Gateway Pattern:** Single entry point for client applications

1.2 Scalability Requirements

- **Current:** 10 facilities (Boston, NY, Philadelphia)
- **2-Year Target:** 14-16 facilities in same regions
- **Growth Pattern:** 40% increase in 2 years
- **Design for:** 20+ facilities within 5 years

2. MICROSERVICES IDENTIFICATION

Core Microservices

Service 1: Parking Operations Service

Responsibilities:

- Manage parking facilities, levels, and slots
- Handle reservations (30 days advance)
- Auto-assign slots (entrance-nearest + load balancing)
- Track real-time occupancy
- Enforce grace periods (15 minutes) and max duration (72 hours)
- Manage special slot types (Handicap, Compact, Oversized, Motorcycle, Carpool)
- Generate parking sessions

Database: `parking_operations_db`

- Tables: facilities, levels, slots, reservations, parking_sessions, vehicles

Key Entities:

- ParkingFacility (Aggregate Root)
- ParkingLevel
- ParkingSlot (with subtypes)
- Reservation
- ParkingSession

Service 2: EV Charging Service

Responsibilities:

- Manage charging stations (8-16 per facility)
- Handle charging sessions with authorization (App/RFID)
- Support target SOC (State of Charge) configuration
- Track real-time station health
- Monitor energy consumption (kWh, power, SOC, errors)
- Enforce 15-minute grace period after charge completion
- Calculate idle fees
- Support remote session control

Database: `ev_charging_db`

- Tables: charging_stations, charging_sessions, station_health_logs, connector_compatibility

Key Entities:

- ChargingStation (Aggregate Root)
- ChargingSession
- ElectricVehicle
- StationHealthLog

Supporting Microservices

Service 3: Customer Management Service

Responsibilities:

- Manage customer accounts (Individual, Corporate, Fleet, Guest)
- Handle vehicle registrations (multiple per account)
- Manage membership tiers (Guest, Basic, Silver, Gold, Platinum, Corporate)
- Support corporate accounts (departmental billing, usage reports)
- Manage payment methods
- Handle guest checkout and pay-by-plate

Database: `customer_management_db`

- Tables: customers, customer_vehicles, memberships, corporate_accounts, departments, payment_methods

Key Entities:

- Customer (Aggregate Root)
- CorporateAccount
- Membership
- CustomerVehicle

Service 4: Billing Service

Responsibilities:

- Calculate parking fees (hourly + time-banded + peak rates)
- Calculate EV charging fees (per kWh + idle fees)
- Apply membership discounts
- Enforce rate caps (Daily $25, Weekly $120, Monthly $300)
- Generate invoices (separate parking and charging)
- Process payments (multiple methods)
- Handle failed payments (invoice + exit hold)
- Corporate departmental billing
- 5-year data retention

Database: `billing_db`

- Tables: invoices, invoice_line_items, payments, payment_methods, pricing_rules, rate_caps

Key Entities:

- Invoice (Aggregate Root)
- ParkingFeeCalculator (Domain Service)
- ChargingFeeCalculator (Domain Service)
- PaymentTransaction

Service 5: Facility Management Service

Responsibilities:

- Configure facilities (10+ garages)
- Manage capacity settings
- Configure special slots (ADA, family, motorcycle, etc.)
- Schedule maintenance
- Manage gate hardware integration (LPR + barriers)
- Configure operating hours and overnight rules

Database: `facility_management_db`

- Tables: facility_configs, capacity_settings, maintenance_schedules, hardware_configs

Key Entities:

- FacilityConfiguration (Aggregate Root)
- CapacitySettings
- MaintenanceSchedule

Integration & Platform Microservices

Service 6: Notification Service (Generic)

Responsibilities:

- Send email, SMS, push notifications
- Grace period warnings
- Charge completion alerts
- Payment notifications
- Reservation confirmations

Technology: Can use third-party service (Twilio, SendGrid)

Service 7: Integration Hub Service: How do we implement integrations as a microservice?

Responsibilities:

- Google Maps API integration (real-time availability)
- Apple Maps API integration
- Payment gateway integration (cards, Apple/Google Pay, PayPal)
- Existing CRM integration
- Gate hardware integration (LPR + QR readers)

Service 8: Analytics & Reporting Service

Responsibilities:

- Daily occupancy reports
- Revenue reports (by facility, period)
- Charging statistics
- Energy consumption reports
- Peak hour analysis
- Corporate usage reports
- Fleet reports

Database: `analytics_db` (Read-optimized, denormalized)

- Populated via Event Sourcing from other services

---

3. SERVICE APIS & ENDPOINTS

3.1 Parking Operations Service API

Base URL: `/api/v1/parking`

Public-Facing Endpoints:

```
# Facilities
GET    /facilities
  → List all facilities with real-time availability
  → Query params: city, hasEVCharging
  → Response: List<FacilityAvailability>

GET    /facilities/{facilityId}
```

```
  → Get facility details and current availability
  → Response: FacilityDetails

GET    /facilities/{facilityId}/availability
  → Real-time slot availability by type
  → Response: { regular: 45, ev: 12, handicap: 3, ... }

# Reservations
POST   /reservations
  → Create reservation (up to 30 days advance)
  → Body: { customerId, facilityId, slotType, startTime, duration }
  → Response: Reservation

GET    /reservations/{reservationId}
  → Get reservation details
  → Response: Reservation

PUT    /reservations/{reservationId}
  → Modify reservation (if within policy)
  → Body: { startTime?, duration? }
  → Response: Reservation

DELETE /reservations/{reservationId}
  → Cancel reservation
  → Response: 204 No Content

GET    /customers/{customerId}/reservations
  → List customer's active reservations
  → Response: List<Reservation>

# Parking Sessions
POST   /sessions/entry
  → Record vehicle entry (via LPR or QR)
  → Body: { facilityId, vehicleId, registrationNumber, reservationId? }
  → Response: { sessionId, assignedSlot, entryTime, exitDeadline }

POST   /sessions/{sessionId}/exit
  → Record vehicle exit
  → Body: { exitTime }
  → Response: ParkingSession (with fee calculation)

GET    /sessions/{sessionId}
  → Get session details
  → Response: ParkingSession

POST   /sessions/{sessionId}/payment-initiated
  → Mark payment initiated (starts 15-min grace period)
  → Response: { gracePeriodEnd }

# Slot Management
GET    /facilities/{facilityId}/slots
  → List all slots with status
  → Query params: slotType, status, level
  → Response: List<Slot>
```

```
PUT    /slots/{slotId}/maintenance
  → Mark slot for maintenance
  → Response: Slot

PUT    /slots/{slotId}/available
  → Clear maintenance, mark available
  → Response: Slot
```

Service-to-Service Endpoints:

```
GET    /internal/slots/{slotId}
  → Get slot details for other services

POST   /internal/slots/{slotId}/reserve
  → Reserve slot (called by Reservation Service)

POST   /internal/facilities/{facilityId}/occupancy-changed
  → Notify occupancy change (triggers Maps API update)
```

3.2 EV Charging Service API

Base URL: `/api/v1/charging`

Public-Facing Endpoints:

```
# Stations
GET    /stations
  → List charging stations
  → Query params: facilityId, connectorType, powerLevel, status
  → Response: List<ChargingStation>

GET    /stations/{stationId}
  → Get station details and availability
  → Response: ChargingStationDetails

GET    /stations/{stationId}/availability
  → Check if station is available
  → Response: { available: boolean, estimatedWaitTime?: int }

# Charging Sessions
POST   /sessions
  → Start charging session
  → Body: { stationId, vehicleId, authMethod, targetSOC? }
  → Response: ChargingSession

GET    /sessions/{sessionId}
  → Get session details and progress
```

```
  → Response: ChargingSession (with real-time SOC, kWh, power)

PUT    /sessions/{sessionId}/target-soc
  → Update target SOC during charging
  → Body: { targetSOC }
  → Response: ChargingSession

POST   /sessions/{sessionId}/stop
  → Stop charging (remote control via app)
  → Response: ChargingSession (final stats)

GET    /customers/{customerId}/sessions
  → List customer's charging history
  → Query params: startDate, endDate, status
  → Response: List<ChargingSession>

# Station Health
GET    /stations/{stationId}/health
  → Get station health status
  → Response: { status, lastCheck, errorCount, predictiveMaintenance? }

POST   /stations/{stationId}/maintenance
  → Report station issue or schedule maintenance
  → Body: { issue, scheduledDate? }
  → Response: MaintenanceTicket
```

Service-to-Service Endpoints:

```
GET    /internal/stations/{stationId}/session
  → Get current session for station

POST   /internal/sessions/{sessionId}/update-progress
  → Update session progress (from station hardware)
  → Body: { currentSOC, powerDelivered, timestamp }

POST   /internal/sessions/{sessionId}/grace-period-expired
  → Notify idle fee period started
```

3.3 Customer Management Service API

Base URL: `/api/v1/customers`

Public-Facing Endpoints:

```
# Customers
POST   /customers
  → Register new customer
  → Body: { name, email, phone, accountType }
```

```
  → Response: Customer

GET    /customers/{customerId}
  → Get customer profile
  → Response: Customer

PUT    /customers/{customerId}
  → Update customer profile
  → Body: { name?, email?, phone?, preferences? }
  → Response: Customer

# Vehicles
POST   /customers/{customerId}/vehicles
  → Register vehicle
  → Body: { registrationNumber, make, model, color, vehicleType, isElectric, ... }
  → Response: CustomerVehicle

GET    /customers/{customerId}/vehicles
  → List customer's vehicles
  → Response: List<CustomerVehicle>

DELETE /customers/{customerId}/vehicles/{vehicleId}
  → Remove vehicle
  → Response: 204 No Content

# Membership
PUT    /customers/{customerId}/membership
  → Upgrade/change membership tier
  → Body: { tier }
  → Response: Membership

GET    /customers/{customerId}/membership
  → Get membership details
  → Response: Membership

# Corporate Accounts
POST   /corporate-accounts
  → Create corporate account
  → Body: { companyName, taxId, billingAddress }
  → Response: CorporateAccount

POST   /corporate-accounts/{corporateId}/users
  → Add user to corporate account
  → Body: { customerId, department, costCenter }
  → Response: 201 Created

GET    /corporate-accounts/{corporateId}/users
  → List corporate users
  → Response: List<CorporateUser>

GET    /corporate-accounts/{corporateId}/usage-report
  → Generate usage report
  → Query params: department?, startDate, endDate
  → Response: UsageReport
```

```
# Payment Methods
POST   /customers/{customerId}/payment-methods
  → Add payment method
  → Body: { type, ... }
  → Response: PaymentMethod

PUT    /customers/{customerId}/payment-methods/{methodId}/default
  → Set default payment method
  → Response: 204 No Content

# Guest Checkout
POST   /guest-checkout
  → Create guest session (pay-by-plate)
  → Body: { registrationNumber, email? }
  → Response: { guestSessionId, checkoutUrl }
```

Service-to-Service Endpoints:

```
GET    /internal/customers/{customerId}
  → Get customer for other services

GET    /internal/customers/by-vehicle/{registrationNumber}
  → Find customer by vehicle registration

GET    /internal/customers/{customerId}/membership-tier
  → Get membership tier for billing discounts
```

3.4 Billing Service API

Base URL: `/api/v1/billing`

Public-Facing Endpoints:

```
# Invoices
GET    /invoices/{invoiceId}
  → Get invoice details
  → Response: Invoice

GET    /customers/{customerId}/invoices
  → List customer invoices
  → Query params: startDate, endDate, status
  → Response: List<Invoice>

POST   /invoices/{invoiceId}/pay
  → Pay invoice
  → Body: { paymentMethodId }
  → Response: PaymentTransaction
```

```
# Fee Calculation (Preview)
POST   /calculate-parking-fee
  → Calculate parking fee before exit
  → Body: { sessionId, exitTime }
  → Response: { baseFee, discount, tax, total, breakdown }

POST   /calculate-charging-fee
  → Calculate charging fee
  → Body: { sessionId }
  → Response: { energyFee, idleFee, total, breakdown }

# Corporate Billing
GET    /corporate-accounts/{corporateId}/invoices
  → Get corporate invoices
  → Query params: month, department?
  → Response: List<CorporateInvoice>

GET    /corporate-accounts/{corporateId}/departmental-report
  → Departmental billing breakdown
  → Query params: month, department
  → Response: DepartmentalReport
```

Service-to-Service Endpoints:

```
POST   /internal/parking-session-fee
  → Calculate fee for parking session (called by Parking Service)
  → Body: ParkingSession
  → Response: Money

POST   /internal/charging-session-fee
  → Calculate fee for charging session (called by Charging Service)
  → Body: ChargingSession
  → Response: Money

POST   /internal/generate-invoice
  → Generate invoice for sessions
  → Body: { customerId, parkingSessionIds, chargingSessionIds }
  → Response: Invoice

POST   /internal/payment-failed
  → Notify of failed payment (triggers exit hold)
  → Body: { invoiceId, customerId }
```

3.5 Facility Management Service API

Base URL: `/api/v1/facilities`

Admin-Only Endpoints:

```
# Facility Configuration
POST   /facilities
  → Create new facility
  → Body: { name, address, city, capacity }
  → Response: Facility

PUT    /facilities/{facilityId}/config
  → Update facility configuration
  → Body: { operatingHours?, maxDuration?, allowsOvernight?, ... }
  → Response: FacilityConfig

POST   /facilities/{facilityId}/levels
  → Add level to facility
  → Body: { levelNumber, regularSlots, evSlots, ... }
  → Response: Level

# Capacity Management
PUT    /facilities/{facilityId}/capacity
  → Update capacity settings
  → Body: { totalSlots, slotTypeDistribution }
  → Response: CapacitySettings

# Maintenance
POST   /facilities/{facilityId}/maintenance
  → Schedule facility maintenance
  → Body: { scheduledDate, duration, reason }
  → Response: MaintenanceSchedule

GET    /facilities/{facilityId}/maintenance
  → Get maintenance schedule
  → Response: List<MaintenanceSchedule>

# Hardware Integration
PUT    /facilities/{facilityId}/gate-hardware
  → Configure gate hardware (LPR + barriers)
  → Body: { lprModel, barrierModel, integrationEndpoint }
  → Response: HardwareConfig
```

Service-to-Service Endpoints:

```
GET    /internal/facilities/{facilityId}/config
  → Get facility configuration for other services

GET    /internal/facilities/{facilityId}/operating-hours
  → Get operating hours for validation
```

3.6 Analytics & Reporting Service API

Base URL: `/api/v1/analytics`

Reporting Endpoints:

```
# Occupancy Reports
GET    /occupancy/daily
  → Daily occupancy report
  → Query params: facilityId?, date
  → Response: DailyOccupancyReport

GET    /occupancy/peak-hours
  → Peak hour analysis
  → Query params: facilityId?, startDate, endDate
  → Response: PeakHourAnalysis

# Revenue Reports
GET    /revenue/by-facility
  → Revenue breakdown by facility
  → Query params: startDate, endDate
  → Response: FacilityRevenueReport

GET    /revenue/by-period
  → Revenue over time
  → Query params: period (daily/weekly/monthly), startDate, endDate
  → Response: TimeSeries<Revenue>

# Charging Analytics
GET    /charging/statistics
  → Charging usage statistics
  → Query params: facilityId?, startDate, endDate
  → Response: ChargingStatistics

GET    /charging/energy-consumption
  → Energy consumption report
  → Query params: facilityId?, startDate, endDate
  → Response: EnergyConsumptionReport

# Custom Reports
POST   /reports/generate
  → Generate custom report
  → Body: { reportType, filters, groupBy, dateRange }
  → Response: { reportId, status, downloadUrl }

GET    /reports/{reportId}
  → Get generated report
  → Response: Report (CSV/PDF/JSON)
```

4. DATABASE DESIGN PER SERVICE

4.1 Parking Operations Database (`parking_operations_db`)

```sql
-- Facilities
CREATE TABLE facilities (
    facility_id UUID PRIMARY KEY,
    facility_code VARCHAR(20) UNIQUE NOT NULL,
    name VARCHAR(200) NOT NULL,
    city VARCHAR(50) NOT NULL,
    address_street VARCHAR(200),
    address_city VARCHAR(100),
    address_state VARCHAR(50),
    address_zip VARCHAR(20),
    operating_hours_open TIME,
    operating_hours_close TIME,
    is_24_hours BOOLEAN DEFAULT FALSE,
    allows_overnight BOOLEAN DEFAULT FALSE,
    overnight_rate DECIMAL(10,2),
    max_duration_hours INT DEFAULT 72,
    status VARCHAR(20) DEFAULT 'OPEN',
    has_lpr BOOLEAN DEFAULT TRUE,
    has_mobile_qr BOOLEAN DEFAULT TRUE,
    gate_hardware_model VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Levels
CREATE TABLE levels (
    level_id UUID PRIMARY KEY,
    facility_id UUID NOT NULL REFERENCES facilities(facility_id),
    level_number INT NOT NULL,
    status VARCHAR(20) DEFAULT 'OPEN',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(facility_id, level_number)
);

-- Slots
CREATE TABLE slots (
    slot_id UUID PRIMARY KEY,
    level_id UUID NOT NULL REFERENCES levels(level_id),
    slot_number VARCHAR(20) NOT NULL,
    slot_type VARCHAR(20) NOT NULL, -- REGULAR, EV, HANDICAP, COMPACT, OVERSIZED,
MOTORCYCLE, CARPOOL
    status VARCHAR(20) DEFAULT 'AVAILABLE', -- AVAILABLE, OCCUPIED, RESERVED,
MAINTENANCE
    current_vehicle_id UUID,
    current_reservation_id UUID,
    entry_time TIMESTAMP,
    exit_deadline TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(level_id, slot_number)
);

CREATE INDEX idx_slots_status ON slots(status);
```

```
CREATE INDEX idx_slots_type_status ON slots(slot_type, status);

-- Reservations
CREATE TABLE reservations (
    reservation_id UUID PRIMARY KEY,
    facility_id UUID NOT NULL REFERENCES facilities(facility_id),
    customer_id UUID NOT NULL,
    vehicle_id UUID NOT NULL,
    slot_type VARCHAR(20) NOT NULL,
    reservation_time TIMESTAMP NOT NULL,
    start_time TIMESTAMP NOT NULL,
    duration_minutes INT NOT NULL,
    status VARCHAR(20) DEFAULT 'PENDING', -- PENDING, CONFIRMED, ACTIVE,
COMPLETED, CANCELLED, NO_SHOW
    assigned_slot_id UUID REFERENCES slots(slot_id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_reservations_customer ON reservations(customer_id);
CREATE INDEX idx_reservations_start_time ON reservations(start_time);
CREATE INDEX idx_reservations_status ON reservations(status);

-- Parking Sessions
CREATE TABLE parking_sessions (
    session_id UUID PRIMARY KEY,
    facility_id UUID NOT NULL REFERENCES facilities(facility_id),
    slot_id UUID NOT NULL REFERENCES slots(slot_id),
    vehicle_id UUID NOT NULL,
    registration_number VARCHAR(20) NOT NULL,
    customer_id UUID,
    reservation_id UUID REFERENCES reservations(reservation_id),
    entry_time TIMESTAMP NOT NULL,
    exit_time TIMESTAMP,
    duration_minutes INT,
    was_peak_hours BOOLEAN DEFAULT FALSE,
    grace_period_started TIMESTAMP,
    parking_fee DECIMAL(10,2),
    status VARCHAR(20) DEFAULT 'ACTIVE', -- ACTIVE, COMPLETED, GRACE_PERIOD,
OVERSTAY
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_parking_sessions_vehicle ON parking_sessions(vehicle_id);
CREATE INDEX idx_parking_sessions_customer ON parking_sessions(customer_id);
CREATE INDEX idx_parking_sessions_entry_time ON parking_sessions(entry_time);
CREATE INDEX idx_parking_sessions_status ON parking_sessions(status);

-- Vehicles (cached from Customer Service)
CREATE TABLE vehicles (
    vehicle_id UUID PRIMARY KEY,
    registration_number VARCHAR(20) UNIQUE NOT NULL,
    make VARCHAR(100),
```

```
    model VARCHAR(100),
    color VARCHAR(50),
    vehicle_type VARCHAR(20), -- CAR, MOTORCYCLE, TRUCK, BUS
    is_electric BOOLEAN DEFAULT FALSE,
    last_synced TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_vehicles_registration ON vehicles(registration_number);
```

4.2 EV Charging Database (ev_charging_db)

```
-- Charging Stations
CREATE TABLE charging_stations (
    station_id UUID PRIMARY KEY,
    station_code VARCHAR(30) UNIQUE NOT NULL,
    facility_id UUID NOT NULL,
    location_level INT,
    location_zone VARCHAR(20),
    connector_type VARCHAR(20) NOT NULL, -- TYPE2, CCS, CHAdeMO
    power_level VARCHAR(20) NOT NULL, -- LEVEL2, DC_FAST, ULTRA_FAST
    max_power_kw DECIMAL(6,2) NOT NULL,
    status VARCHAR(20) DEFAULT 'AVAILABLE', -- AVAILABLE, CHARGING, OFFLINE,
MAINTENANCE, RESERVED
    health_status VARCHAR(20) DEFAULT 'HEALTHY', -- HEALTHY, DEGRADED, FAULTY
    total_energy_dispensed_kwh DECIMAL(12,2) DEFAULT 0,
    total_sessions INT DEFAULT 0,
    error_count INT DEFAULT 0,
    last_error_time TIMESTAMP,
    last_maintenance_date TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_stations_facility ON charging_stations(facility_id);
CREATE INDEX idx_stations_status ON charging_stations(status);
CREATE INDEX idx_stations_connector ON charging_stations(connector_type);

-- Charging Sessions
CREATE TABLE charging_sessions (
    session_id UUID PRIMARY KEY,
    station_id UUID NOT NULL REFERENCES charging_stations(station_id),
    vehicle_id UUID NOT NULL,
    registration_number VARCHAR(20) NOT NULL,
    customer_id UUID NOT NULL,
    auth_method VARCHAR(20) NOT NULL, -- MOBILE_APP, RFID_CARD
    start_time TIMESTAMP NOT NULL,
    end_time TIMESTAMP,
    charging_duration_minutes INT,
    idle_duration_minutes INT,
    start_soc_percent DECIMAL(5,2) NOT NULL,
    current_soc_percent DECIMAL(5,2),
```

```
    target_soc_percent DECIMAL(5,2),
    final_soc_percent DECIMAL(5,2),
    energy_consumed_kwh DECIMAL(8,3),
    peak_power_kw DECIMAL(6,2),
    average_power_kw DECIMAL(6,2),
    status VARCHAR(20) DEFAULT 'IN_PROGRESS', -- IN_PROGRESS, TARGET_REACHED,
COMPLETED, INTERRUPTED, FAILED
    error_code VARCHAR(50),
    error_message TEXT,
    charging_fee DECIMAL(10,2),
    idle_fee DECIMAL(10,2),
    grace_period_end TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_sessions_station ON charging_sessions(station_id);
CREATE INDEX idx_sessions_vehicle ON charging_sessions(vehicle_id);
CREATE INDEX idx_sessions_customer ON charging_sessions(customer_id);
CREATE INDEX idx_sessions_start_time ON charging_sessions(start_time);
CREATE INDEX idx_sessions_status ON charging_sessions(status);

-- Station Health Logs
CREATE TABLE station_health_logs (
    log_id UUID PRIMARY KEY,
    station_id UUID NOT NULL REFERENCES charging_stations(station_id),
    check_time TIMESTAMP NOT NULL,
    health_status VARCHAR(20) NOT NULL,
    power_output_kw DECIMAL(6,2),
    voltage DECIMAL(6,2),
    current_amps DECIMAL(6,2),
    temperature_celsius DECIMAL(5,2),
    error_codes TEXT[],
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_health_logs_station_time ON station_health_logs(station_id,
check_time DESC);

-- Electric Vehicles (cached from Customer Service)
CREATE TABLE electric_vehicles (
    vehicle_id UUID PRIMARY KEY,
    registration_number VARCHAR(20) UNIQUE NOT NULL,
    make VARCHAR(100),
    model VARCHAR(100),
    battery_capacity_kwh DECIMAL(6,2),
    connector_type VARCHAR(20),
    max_charging_rate_kw DECIMAL(6,2),
    last_synced TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_ev_registration ON electric_vehicles(registration_number);
```

4.3 Customer Management Database (`customer_management_db`)

```
-- Customers
CREATE TABLE customers (
    customer_id UUID PRIMARY KEY,
    account_type VARCHAR(20) NOT NULL, -- INDIVIDUAL, CORPORATE, FLEET, GUEST
    first_name VARCHAR(100),
    last_name VARCHAR(100),
    email VARCHAR(255) UNIQUE,
    phone VARCHAR(20),
    is_guest BOOLEAN DEFAULT FALSE,
    guest_session_id UUID,
    corporate_id UUID,
    department VARCHAR(100),
    cost_center VARCHAR(50),
    approval_limit DECIMAL(10,2),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_customers_email ON customers(email);
CREATE INDEX idx_customers_corporate ON customers(corporate_id);

-- Customer Vehicles
CREATE TABLE customer_vehicles (
    vehicle_id UUID PRIMARY KEY,
    customer_id UUID NOT NULL REFERENCES customers(customer_id),
    registration_number VARCHAR(20) UNIQUE NOT NULL,
    make VARCHAR(100),
    model VARCHAR(100),
    color VARCHAR(50),
    vehicle_type VARCHAR(20), -- CAR, MOTORCYCLE, TRUCK, BUS
    is_electric BOOLEAN DEFAULT FALSE,
    battery_capacity_kwh DECIMAL(6,2),
    connector_type VARCHAR(20),
    max_charging_rate_kw DECIMAL(6,2),
    is_primary BOOLEAN DEFAULT FALSE,
    registration_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_customer_vehicles_customer ON customer_vehicles(customer_id);
CREATE INDEX idx_customer_vehicles_registration ON
customer_vehicles(registration_number);

-- Memberships
CREATE TABLE memberships (
    membership_id UUID PRIMARY KEY,
    customer_id UUID NOT NULL REFERENCES customers(customer_id) UNIQUE,
    tier VARCHAR(20) NOT NULL, -- GUEST, BASIC, SILVER, GOLD, PLATINUM, CORPORATE
    start_date TIMESTAMP NOT NULL,
```

```
    expiry_date TIMESTAMP,
    discount_percent DECIMAL(5,2) DEFAULT 0,
    has_priority_access BOOLEAN DEFAULT FALSE,
    benefits JSONB,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_memberships_customer ON memberships(customer_id);
CREATE INDEX idx_memberships_tier ON memberships(tier);

-- Corporate Accounts
CREATE TABLE corporate_accounts (
    corporate_id UUID PRIMARY KEY,
    company_name VARCHAR(200) NOT NULL,
    tax_id VARCHAR(50),
    billing_address TEXT,
    monthly_billing_cycle BOOLEAN DEFAULT TRUE,
    credit_limit DECIMAL(12,2),
    current_balance DECIMAL(12,2) DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Departments
CREATE TABLE departments (
    department_id UUID PRIMARY KEY,
    corporate_id UUID NOT NULL REFERENCES corporate_accounts(corporate_id),
    department_name VARCHAR(100) NOT NULL,
    cost_center VARCHAR(50),
    budget DECIMAL(12,2),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(corporate_id, department_name)
);

-- Payment Methods
CREATE TABLE payment_methods (
    payment_method_id UUID PRIMARY KEY,
    customer_id UUID NOT NULL REFERENCES customers(customer_id),
    method_type VARCHAR(20) NOT NULL, -- CREDIT_CARD, DEBIT_CARD, PAYPAL, PREPAID,
CORPORATE_BILLING
    is_default BOOLEAN DEFAULT FALSE,
    card_last4 VARCHAR(4),
    card_brand VARCHAR(20),
    card_expiry_month INT,
    card_expiry_year INT,
    billing_address TEXT,
    payment_gateway_token VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_payment_methods_customer ON payment_methods(customer_id);
```

4.4 Billing Database (billing_db)

```
-- Invoices
CREATE TABLE invoices (
    invoice_id UUID PRIMARY KEY,
    customer_id UUID NOT NULL,
    account_type VARCHAR(20) NOT NULL,
    parking_subtotal DECIMAL(10,2) DEFAULT 0,
    charging_subtotal DECIMAL(10,2) DEFAULT 0,
    subtotal DECIMAL(10,2) NOT NULL,
    discount_percent DECIMAL(5,2) DEFAULT 0,
    discount_amount DECIMAL(10,2) DEFAULT 0,
    tax_percent DECIMAL(5,2) DEFAULT 0,
    tax_amount DECIMAL(10,2) DEFAULT 0,
    total_amount DECIMAL(10,2) NOT NULL,
    issue_date TIMESTAMP NOT NULL,
    due_date TIMESTAMP NOT NULL,
    status VARCHAR(20) DEFAULT 'DRAFT', -- DRAFT, ISSUED, PAID, OVERDUE, CANCELLED
    payment_method_id UUID,
    paid_date TIMESTAMP,
    department VARCHAR(100),
    cost_center VARCHAR(50),
    retention_end_date TIMESTAMP NOT NULL, -- issue_date + 5 years
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_invoices_customer ON invoices(customer_id);
CREATE INDEX idx_invoices_status ON invoices(status);
CREATE INDEX idx_invoices_issue_date ON invoices(issue_date);
CREATE INDEX idx_invoices_retention ON invoices(retention_end_date);

-- Invoice Line Items
CREATE TABLE invoice_line_items (
    line_item_id UUID PRIMARY KEY,
    invoice_id UUID NOT NULL REFERENCES invoices(invoice_id),
    item_type VARCHAR(20) NOT NULL, -- PARKING, CHARGING, IDLE_FEE, LOST_TICKET,
OVERSTAY
    session_id UUID, -- parking_session_id or charging_session_id
    description TEXT NOT NULL,
    quantity DECIMAL(10,3) DEFAULT 1,
    unit_price DECIMAL(10,2) NOT NULL,
    total DECIMAL(10,2) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_line_items_invoice ON invoice_line_items(invoice_id);

-- Payment Transactions
CREATE TABLE payment_transactions (
    transaction_id UUID PRIMARY KEY,
```

```
    invoice_id UUID NOT NULL REFERENCES invoices(invoice_id),
    customer_id UUID NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    payment_method VARCHAR(20) NOT NULL,
    transaction_time TIMESTAMP NOT NULL,
    status VARCHAR(20) NOT NULL, -- SUCCESS, FAILED, PENDING, REFUNDED
    gateway_transaction_id VARCHAR(255),
    failure_reason TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_transactions_invoice ON payment_transactions(invoice_id);
CREATE INDEX idx_transactions_customer ON payment_transactions(customer_id);
CREATE INDEX idx_transactions_time ON payment_transactions(transaction_time);

-- Pricing Rules (Michael's rates)
CREATE TABLE pricing_rules (
    rule_id UUID PRIMARY KEY,
    rule_type VARCHAR(20) NOT NULL, -- PARKING_HOURLY, CHARGING_KWH, IDLE_FEE
    slot_type VARCHAR(20), -- For parking: REGULAR, EV, HANDICAP, COMPACT
    power_level VARCHAR(20), -- For charging: LEVEL2, DC_FAST, ULTRA_FAST
    base_rate DECIMAL(10,4) NOT NULL,
    peak_multiplier DECIMAL(5,2) DEFAULT 1.0,
    effective_from TIMESTAMP NOT NULL,
    effective_until TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Rate Caps (Michael's caps)
CREATE TABLE rate_caps (
    cap_id UUID PRIMARY KEY,
    cap_type VARCHAR(20) NOT NULL, -- DAILY, WEEKLY, MONTHLY
    max_amount DECIMAL(10,2) NOT NULL,
    effective_from TIMESTAMP NOT NULL,
    effective_until TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Insert Michael's pricing rules
INSERT INTO pricing_rules (rule_id, rule_type, slot_type, base_rate,
effective_from) VALUES
(gen_random_uuid(), 'PARKING_HOURLY', 'REGULAR', 3.00, CURRENT_TIMESTAMP),
(gen_random_uuid(), 'PARKING_HOURLY', 'EV', 4.00, CURRENT_TIMESTAMP),
(gen_random_uuid(), 'PARKING_HOURLY', 'HANDICAP', 1.50, CURRENT_TIMESTAMP),
(gen_random_uuid(), 'PARKING_HOURLY', 'COMPACT', 2.00, CURRENT_TIMESTAMP);

INSERT INTO pricing_rules (rule_id, rule_type, power_level, base_rate,
effective_from) VALUES
(gen_random_uuid(), 'CHARGING_KWH', 'LEVEL2', 0.35, CURRENT_TIMESTAMP),
(gen_random_uuid(), 'CHARGING_KWH', 'DC_FAST', 0.50, CURRENT_TIMESTAMP),
(gen_random_uuid(), 'CHARGING_KWH', 'ULTRA_FAST', 0.50, CURRENT_TIMESTAMP);

INSERT INTO rate_caps (cap_id, cap_type, max_amount, effective_from) VALUES
(gen_random_uuid(), 'DAILY', 25.00, CURRENT_TIMESTAMP),
```

```
(gen_random_uuid(), 'WEEKLY', 120.00, CURRENT_TIMESTAMP),
(gen_random_uuid(), 'MONTHLY', 300.00, CURRENT_TIMESTAMP);
```

4.5 Analytics Database (`analytics_db`)

```
-- Denormalized for read-heavy analytics

-- Daily Occupancy Facts
CREATE TABLE daily_occupancy_facts (
    fact_id UUID PRIMARY KEY,
    date DATE NOT NULL,
    facility_id UUID NOT NULL,
    facility_name VARCHAR(200),
    city VARCHAR(50),
    total_slots INT NOT NULL,
    avg_occupancy_percent DECIMAL(5,2) NOT NULL,
    peak_occupancy_percent DECIMAL(5,2) NOT NULL,
    peak_occupancy_time TIME,
    total_sessions INT NOT NULL,
    avg_session_duration_minutes INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_occupancy_date_facility ON daily_occupancy_facts(date,
facility_id);

-- Revenue Facts
CREATE TABLE revenue_facts (
    fact_id UUID PRIMARY KEY,
    date DATE NOT NULL,
    facility_id UUID NOT NULL,
    facility_name VARCHAR(200),
    city VARCHAR(50),
    parking_revenue DECIMAL(12,2) DEFAULT 0,
    charging_revenue DECIMAL(12,2) DEFAULT 0,
    idle_fee_revenue DECIMAL(12,2) DEFAULT 0,
    total_revenue DECIMAL(12,2) NOT NULL,
    transaction_count INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_revenue_date_facility ON revenue_facts(date, facility_id);

-- Charging Statistics
CREATE TABLE charging_statistics (
    stat_id UUID PRIMARY KEY,
    date DATE NOT NULL,
    facility_id UUID NOT NULL,
    station_id UUID NOT NULL,
    total_sessions INT NOT NULL,
    total_energy_kwh DECIMAL(10,2) NOT NULL,
```

```
    avg_session_duration_minutes INT,
    avg_energy_per_session_kwh DECIMAL(8,2),
    revenue_generated DECIMAL(10,2) NOT NULL,
    utilization_percent DECIMAL(5,2) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_charging_stats_date_facility ON charging_statistics(date,
facility_id);
```